

AI DRIVEN ADAPTIVE CODING ASSISTANCE AND INTERVIEW PREPARATION PLATFORM

Project ID : 25-26J-364

Project Proposal Report

Sandamal R.T

BSc (Hons) in Information Technology Specializing in Information
Technology

Department of Information Technology Sri Lanka Institute of

Information Technology

Sri Lanka

August 2025

AI DRIVEN ADAPTIVE CODING ASSISTANCE AND INTERVIEW PREPARATION PLATFORM

Project ID : 25-26J-364

Project Proposal Report

Sandamal R.T

BSc (Hons) in Information Technology Specializing in
Information

Department of Information Technology Sri Lanka Institute of

Information Technology

Sri Lanka

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
Sandamal R.T	IT22606860	

The supervisor/s should certify the proposal report with the following declaration.

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor:



Date: 31/08/2025

ABSTRACT

The rapid expansion of programming education among university students has highlighted the need for intelligent systems that not only assist in writing code but also enhance its quality and performance. While most AI assisted educational tools primarily focus on error detection and debugging, they often overlook the importance of code optimization and refactoring for long term maintainability.

This research proposes the development of an AI powered code refactoring assistant designed specifically for university students to improve code efficiency, readability, and performance. The system allows students to upload their code, where it will be analyzed for errors, redundancies, and unnecessary components. Unlike conventional debugging tools, this model not only identifies errors but also provides clear explanations for why they occur, enabling students to build a deeper understanding of programming principles.

Furthermore, the system applies advanced refactoring techniques to restructure the code for improved performance, including optimized memory usage and streamlined execution paths. Importantly, it also evaluates the potential risks introduced by the refactored version, giving students a clear understanding of trade offs between performance gains and stability. The AI model will be fine tuned on a curated dataset consisting of source code examples, refactoring patterns, and best practices in programming. Emphasis will be placed on balancing efficiency, accuracy, and cost effectiveness to ensure the solution is scalable and practical for educational institutions.

The anticipated outcome of this study is an intelligent code refactoring tool that not only enhances the coding skills of students but also fosters better software engineering practices by promoting maintainable, efficient, and risk aware code development. Ultimately, this research aims to bridge the gap between academic programming exercises and real world software development standards.

Table of Contents

DECLARATION	2
ABSTRACT	3
LIST OF FIGURES	5
LIST OF TABLES	6
LIST OF ABBREVIATIONS	6
INTRODUCTION.....	6
1.1. BACKGROUND C LITERATURE SURVEY	7
1.2. RESEARCH GAP	8
1.3. RESEARCH PROBLEM	9
OBJECTIVES	10
2.1. Main Objectives.....	10
2.2. Specific Objectives	10
METHODOLOGY.....	11
PROJECT REQUIREMENTS.....	11
3.1.1 Functional Requirements	11
3.1.2 Non-Functional Requirements.....	12
3.1.3 System Requirements	13
3.2. System Overview	13
3.3. Overall System Diagram	14
3.4. Work Breakdown Structure	14
GRANTT CHART.....	15
REFERENCES.....	15

LIST OF FIGURES

Figure 1 System Overview for Code Refactor	17
Figure 2 Opticode Overall System Diagram.....	18
Figure 3 Work Breakdown Structure	19
Figure 4 Grantt Chart	20

LIST OF TABLES

Table 1 List of Abbreviations..... 7

Table 2 Research Gap 11

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
AST	Abstract Syntax Tree

Table 1 List of Abbreviations

INTRODUCTION

Programming has become a core skill for modern university students, particularly in computing and engineering fields. However, while introductory courses help students understand syntax and basic problem solving, a common challenge faced by learners is writing code that is not only functional but also efficient, optimized, and maintainable. Students frequently produce code that contains redundant statements, unnecessary computations, poor memory handling, or inefficient structures. These issues may not always lead to compilation errors but can significantly degrade performance, readability, and long term usability of the software. Such challenges make it difficult for students to transition from basic coding to professional level software engineering, where code quality and optimization are crucial.

Traditional teaching methods primarily focus on error detection and debugging, often overlooking deeper aspects such as code refactoring, performance improvements, and risk analysis after modification. While Integrated Development Environments (IDEs) provide some automated suggestions, they fail to explain why certain refactoring are important, how they impact performance, and what risks they may introduce. As a result, students rarely learn how to evaluate the trade offs between functional correctness, performance, and maintainability.

This gap in education leads to a situation where many students can ‘make code run’ but struggle to produce code that adheres to best practices in software engineering. Recent advances in Artificial Intelligence , especially in machine learning and large language models, offer new opportunities for intelligent programming assistance. AI based tools can analyse student code, identify redundant or inefficient parts, and generate improved versions that adhere to standard refactoring practices.

However, most of the existing AI powered coding assistants, such as those integrated into professional environments, are not specifically designed for educational use. They often prioritize productivity over pedagogy, giving students answers without explaining underlying reasoning. For university students, this approach hinders learning rather than promoting it.

This research addresses the gap by developing an AI driven code refactoring system tailored for university students. The proposed system allows students to upload their code, which is then analyzed not only for syntactic and logical errors but also for structural inefficiencies.

The system highlights unwanted or redundant parts of the code, explains the reasoning behind errors, and suggests optimized solutions. Beyond simple correction, the tool enhances code performance by improving memory usage, execution efficiency, and readability.

Importantly, the system also evaluates the risks introduced by refactoring, such as potential changes in behavior or performance trade offs, and presents them to the student in an understandable format. By integrating pedagogical explanations with automated refactoring, this project aims to bridge the gap between basic coding education and professional software engineering practices. The final outcome is expected to be a system that not only corrects student mistakes but also teaches them

how to write efficient, optimized, and risk aware code. This will prepare students for both academic success and industry readiness, where performance and maintainability are as important as correctness.

1.1. BACKGROUND & LITERATURE SURVEY

Code refactoring is an essential practice in software engineering, aimed at restructuring existing code to improve its readability, maintainability, and performance without altering its external behavior. For university students learning programming, the focus often remains on producing functionally correct solutions, while efficiency and optimization are overlooked.

This leads to redundant code structures, inefficient memory usage, and poorly organized logic that, while functional, can degrade system performance and make the codebase difficult to maintain in the long term. As students transition from academic exercises to professional software development, these shortcomings can hinder their ability to produce high quality, industry-standard software.

Recent advancements in Artificial Intelligence and Machine Learning have created opportunities for intelligent programming assistance, particularly in the domain of code analysis and refactoring. Modern AI driven coding assistants such as GitHub Copilot and ChatGPT are capable of generating, correcting, and even improving code. However, their focus is often on productivity and automation rather than pedagogy, providing students with solutions without adequately explaining the underlying issues or trade offs. For educational contexts, this creates a gap students may fix errors in their code, but they fail to understand how to enhance performance or evaluate risks associated with refactoring.

Several studies have explored AI-assisted tools for code analysis and educational support. For instance, Ahmad et al. proposed an intelligent tutor that provides realtime feedback on students coding assignments, focusing primarily on syntax correction and logic validation. Similarly, Haldar and Ghosh developed a framework that uses static code analysis and ML models to detect software vulnerabilities and inefficiencies. While these systems provide valuable feedback, they are limited to either error detection or security

analysis and do not extend to comprehensive code restructuring with performance optimization.

Another strand of research has investigated automated program repair and optimization. Le et al. introduced a neural program repair model that generates patches for erroneous code, while Chen et al. proposed a reinforcement learning based approach for automated code optimization. These systems demonstrate the feasibility of AI in enhancing code performance, but they lack features such as pedagogical explanations, risk assessment of refactored code, and memory efficiency analysis,

which are crucial in a learning environment.

Despite these advancements, the integration of code refactoring tools specifically tailored for educational use remains limited. Most systems either provide corrections without teaching, or focus only on debugging rather than holistic refactoring. There is a clear need for a system that not only identifies redundant and inefficient code but also provides students with Explanations of errors and inefficiencies, Refactored versions of their code, Insights into performance improvements such as memory optimization, And an assessment of potential risks introduced by the refactoring process.

This research proposes an AI-powered code refactoring system designed to bridge this gap. Unlike conventional debugging or correction tools, the proposed system emphasizes both performance optimization and pedagogy, ensuring that students not only fix their code but also understand why changes are necessary. By doing so, the system supports students in adopting industry level coding practices while improving their ability to write efficient, risk aware, and maintainable programs.

1.2. RESEARCH GAP

In reviewing existing research and tools related to code analysis and refactoring, it is evident that most systems have been designed either for professional developers in industry settings or for basic debugging in educational contexts. Popular AI assisted coding tools, such as GitHub Copilot and static analysis tools like SonarQube, are capable of identifying syntax errors, bugs, and code smells, but they do not explain why these issues occur or how they affect broader aspects of software quality. Furthermore, they generally lack the ability to provide refactoring suggestions that improve readability, maintainability, or memory efficiency. Some systems attempt performance optimization, but this is done without considering trade-offs or risks.

A major limitation of these existing approaches is the absence of features that are crucial for teaching, such as explanations of errors, pedagogical guidance, and interactive feedback.

Students often receive a correction but not the reasoning behind it, which hinders deeper learning. Similarly, none of the reviewed tools offer risk assessment for refactored code, meaning learners are unaware of the potential impacts of performance or readability changes. While prior studies on program repair and optimization [3][4] highlight the potential of automation, their complexity and resource requirements make them unsuitable for widespread academic adoption.

Therefore, there exists a clear research gap in developing an AI powered code

refactoring assistant specifically for education. The proposed system uniquely integrates error detection, refactoring suggestions, explanations, performance and memory optimization, risk assessment, and interactive chatbot-style guidance. By combining these features, it not only helps students fix code but also fosters understanding of best practices in writing optimized, efficient, and maintainable software bridging the gap between functional correctness and real world software engineering standards.

Features	[2]	[3]	[4]	[5]	Optic ode
Error detection and debugging	✓	✓	✓	✓	✓
Providing Refactoring Suggestions	✗	✗	✗	✓	✓
Explanation of <i>why</i> errors/refactors are suggested	✗	✗	✗	✗	✓
Performance optimization (execution speed, efficiency)	✗	✗	✓	✗	✓
Memory optimization (resource usage awareness)	✗	✗	✗	✗	✓
Risk assessment of refactored code (Low/Med/High)	✗	✗	✗	✗	✓
Interactive chatbot-style guidance	✗	✗	✗	✗	✓

Table 2 Research Gap

1.3. RESEARCH PROBLEM

With the rapid increase in the use of programming among university students, the demand for intelligent tools that enhance both code correctness and quality has grown significantly.

While existing AI based systems focus primarily on error detection and debugging, they often neglect the equally important aspect of code refactoring. Students, particularly in their early years of study, tend to write code that is functional but inefficient, difficult to maintain, or poorly structured. This not only affects their academic performance but also prevents them from developing industry-level software engineering practices.

Despite the advancements in AI driven programming assistance, a critical problem remains: students often learn to fix errors without gaining a deeper understanding of how to improve code readability, performance, and maintainability. As a result, their learning process becomes error oriented rather than quality oriented. Current tools rarely explain why a certain refactoring is necessary or what trade offs exist when optimizing code for memory usage, execution speed, or structural clarity.

Moreover, most available solutions are either too advanced for beginners or too costly to be deployed at scale across educational institutions. This raises a key challenge of how to design a cost effective, scalable AI based code refactoring assistant that provides not only automated improvements but also interactive, step by step explanations to guide students in building better coding practices. The significance of

addressing this research problem lies in its potential to transform programming education. By integrating intelligent refactoring guidance with educational objectives, the system will help students write cleaner, more efficient, and maintainable code while also understanding the underlying principles. This not only enhances their academic learning outcomes but also equips them with practical skills required in real world software development.

OBJECTIVES

2.1. Main Objectives

The main objective of this research is to develop an AI powered code refactoring system that not only improves the quality, readability, and efficiency of student-written programs but also provides step by step explanations to enhance their understanding of best coding practices.

The system aims to guide 1st and 2nd year university students toward writing maintainable and industry standard code, bridging the gap between error correction and long term software development skills.

2.2. Specific Objectives

Integration and Training of a Suitable Language Model

- Select and fine tune a language model capable of understanding programming structures and best practices, ensuring accurate code refactoring recommendations.
- Train the model using a dataset composed of programming assignments, coding standards, and refactoring examples to make it adaptable to beginner level code.
- Evaluate the model's performance on different programming languages (e.g., Java, C++, Python) to ensure cross-language support.

Development of Refactoring Guidance Mechanism

- Implement step by step explanations that accompany every refactoring suggestion, enabling students to understand the reasoning behind each change.
- Provide categorized refactoring such as readability improvements, performance optimization, and structural clarity to align with educational goals.
- Design the system to highlight trade offs and alternative approaches, promoting deeper learning.

Interactive and Educational User Experience

- Create a user friendly interface that integrates features such as syntax highlighting, side-by-side code comparison (before and after refactoring), and real time feedback.
- Allow students to interact with the system by accepting, rejecting, or requesting further clarification on suggested changes.

- Incorporate gamified elements (e.g., progress tracking, learning badges) to maintain student engagement.

METHODOLOGY

The proposed system aims to develop an intelligent code refactoring assistant by leveraging advanced Natural Language Processing (NLP) and AI-driven program analysis techniques. The methodology focuses on analysing students' source code, identifying inefficiencies, and applying structured refactoring strategies while maintaining code functionality and readability. To achieve this, the system integrates Hugging Face Transformers, static code analysis tools, and fine-tuned models trained on curated programming datasets. The process begins with code parsing and analysis, where user-submitted code is tokenized and represented as an Abstract Syntax Tree (AST). This representation enables the detection of redundant code segments, unused variables, poor memory allocation, and inefficient control structures. The AI model, fine-tuned on refactoring patterns and best practices, generates optimized suggestions and explanations for each identified issue.

Next, the system applies automated refactoring techniques to restructure the code, focusing on improving performance, memory efficiency, and maintainability without altering the program's output. To ensure educational value, the chatbot not only provides the refactored code but also explains the reasoning behind each modification, helping students understand the trade-offs between readability, efficiency, and execution speed.

The implementation is carried out using Python for AI integration and Java/Python-based static analysis libraries for program inspection. Cloud-based services are used for model deployment, ensuring scalability and efficient response times when multiple students interact with the system simultaneously. Additionally, the system includes a risk assessment module that highlights potential drawbacks of the applied refactoring, enabling students to make informed decisions about whether to adopt the suggested changes. Through this methodology, the system ensures a balance between automated optimization and educational guidance, providing students with an interactive learning experience while instilling industry-level software engineering practices.

PROJECT REQUIREMENTS

3.1.1 Functional Requirements

Functional requirements describe the core operations of the AI code refactoring system and how it interacts with users. The main functional requirements are:

- **System Login**

A secure login mechanism ensures that only authorized students and instructors can access the system. Authentication is managed through unique credentials such as usernames and passwords, with options for recovery and password management.

- **Code Submission and Parsing**

Users can upload or paste their program code into the system. The system parses the submitted code to detect syntax, structural issues, and potential inefficiencies.

- **Code Refactoring Suggestions**

The system identifies areas of the code that can be optimized for readability, memory usage, execution efficiency, and maintainability. Detailed explanations are provided alongside each suggested change.

- **Performance and Risk Analysis**

After applying suggested refactoring, the system evaluates the performance impact and potential risks associated with each modification. Students are informed about trade-offs between optimization and code safety.

- **Interactive Feedback and Learning**

The system provides step-by-step guidance, allowing students to accept, reject, or request clarification on suggested refactoring changes. This promotes a learning-oriented approach rather than mere automation.

- **Progress Tracking**

Students' interaction with the refactoring system is logged, enabling visualization of learning progress and improvement in code quality over time.

3.1.2. Non-Functional Requirements

Non-functional requirements ensure that the system delivers a reliable, secure, and user- friendly experience.

- **Performance**

The system must process submitted code efficiently and provide refactoring suggestions in real-time, even for large programs.

- **Usability**

The interface should be intuitive and easy for students at all skill levels. Features such

as syntax highlighting, side-by-side code comparison, and clear instructions enhance usability.

- Security

Student code and account data must be securely stored and transmitted. Sensitive information should be encrypted, and secure authentication protocols must be implemented.

- Scalability

The system should handle multiple users simultaneously and allow for future expansion, including additional programming languages or more advanced refactoring techniques.

- Compatibility

The system should be accessible via web browsers on desktops and laptops, with potential future support for tablets.

- Reliability

The system must remain available and recover quickly in the event of failures. Regular maintenance and updates are required to ensure smooth operation.

3.1.3 System Requirements Software Requirements

- User End:

Web browser (Chrome, Firefox, Edge) Internet connection

- Developer End:

Python

Hugging Face Transformers library Static code analysis libraries

Database (MongoDB or MySQL for storing user data and logs)

3.2. System Overview

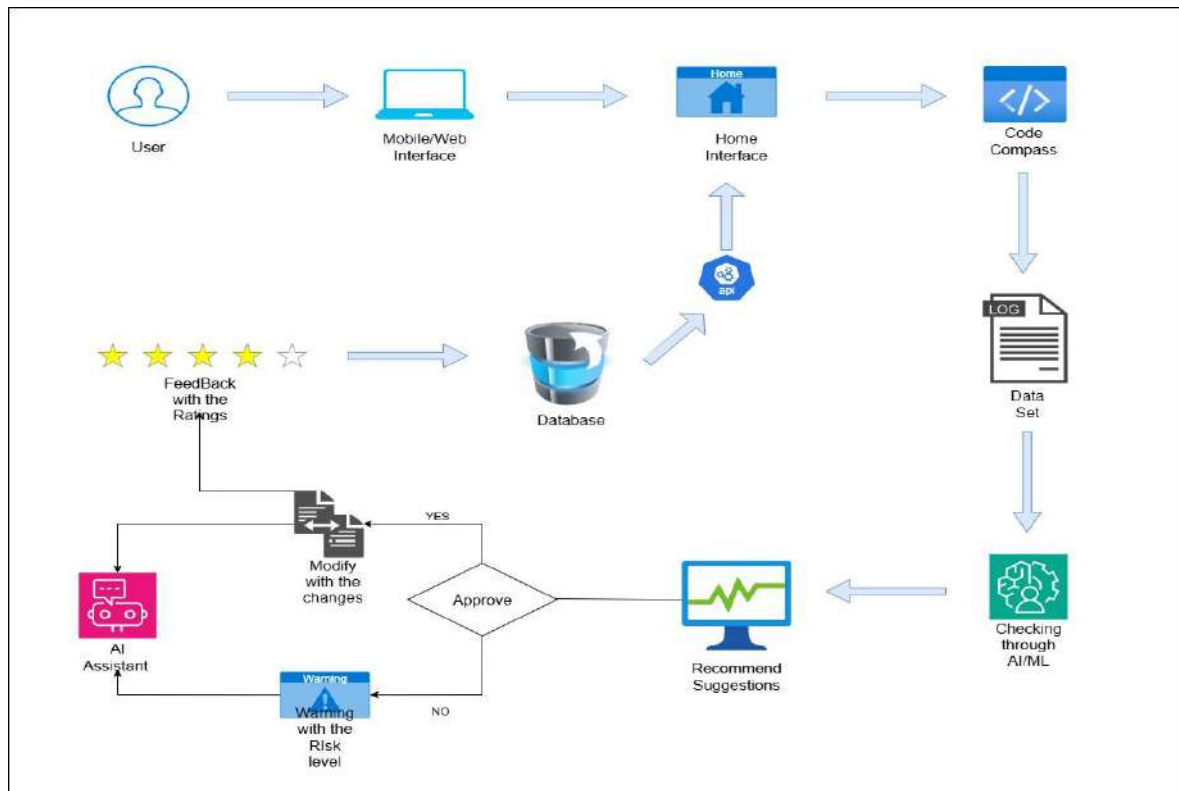


Figure 1 System Overview for Code Refactor

Figure 2 Opticode Overall System DiagramFigure 3 System Overview for Code Refactor

3.3. Overall System Diagram

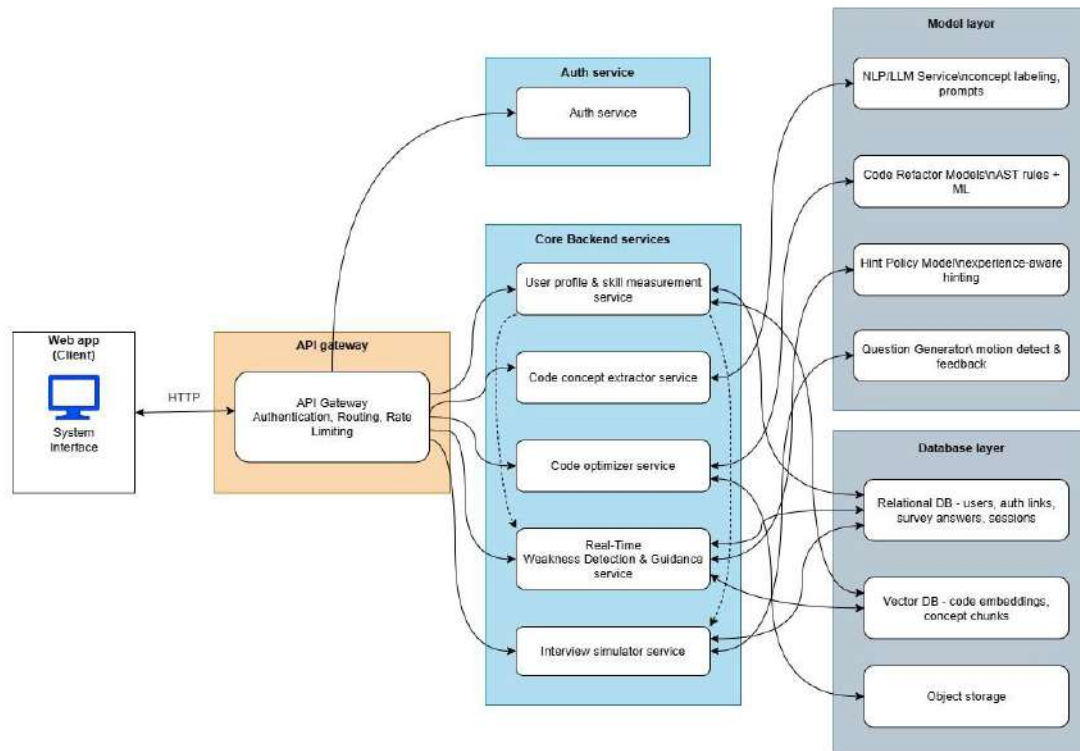


Figure 4 Opticode Overall System Diagram

Figure 5 Work Breakdown StructureFigure 6 Opticode Overall System Diagram

3.4. Work Breakdown Structure

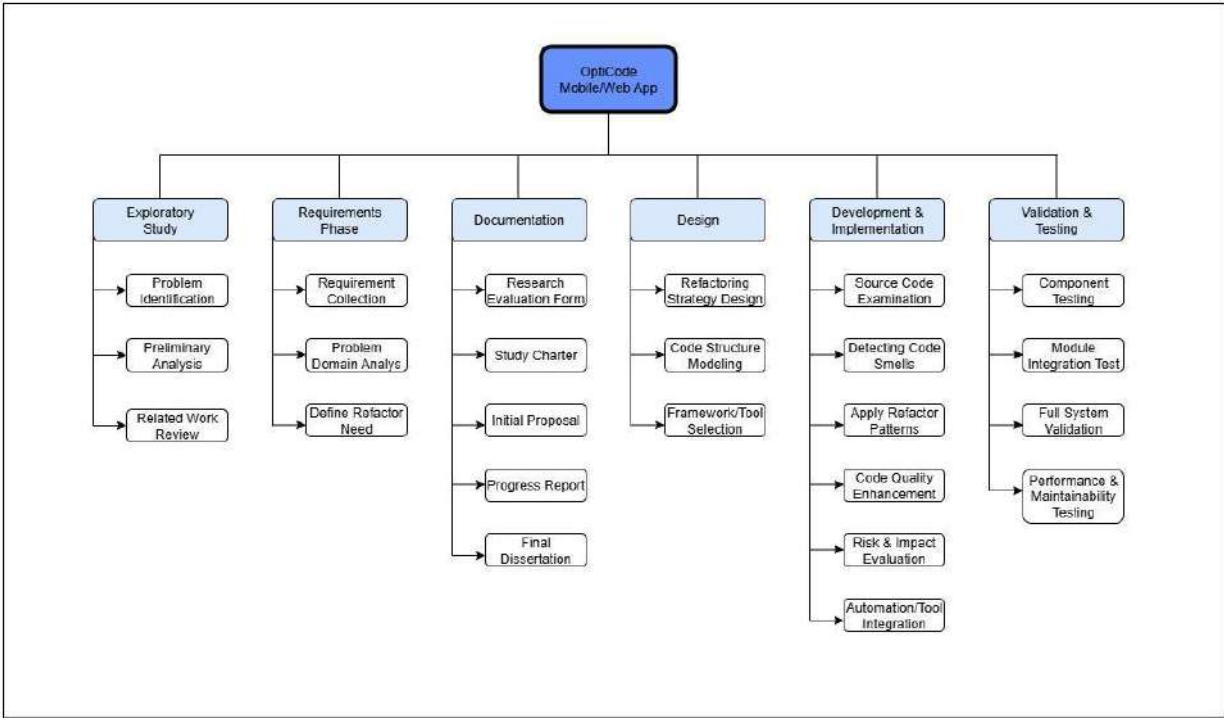


Figure 7 Work Breakdown Structure

Figure 8 Grantt ChartFigure 9 Work Breakdown Structure

GRANTT CHART

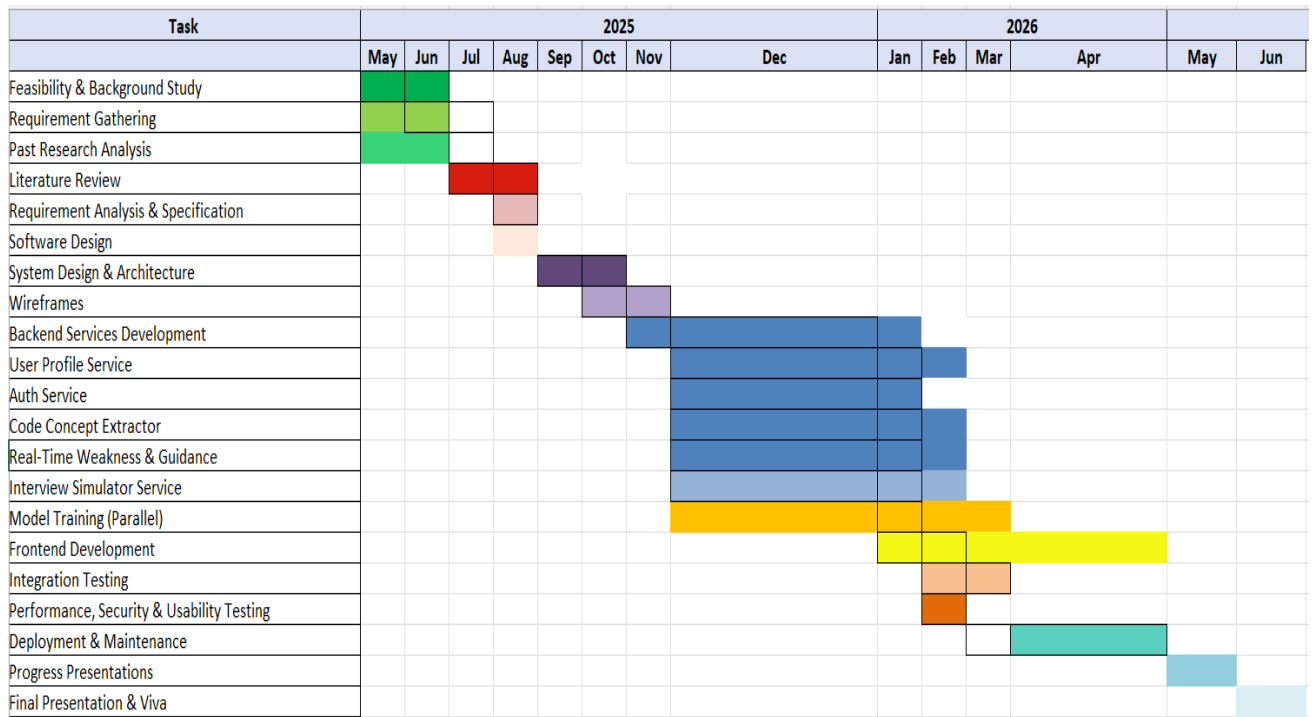


Figure 10 Grantt Chart

REFERENCES

- Abbès, M., Khomh, F., Gueheneuc, Y. G., & Antoniol, G. (2011). *An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension*. In 2011 15th European Conference on Software Maintenance and Reengineering (pp. 181–190). IEEE. <https://doi.org/10.1109/CSMR.2011.24>
- Tsantalis, N., Chaikalis, T., & Chatzigeorgiou, A. (2008). *JDeodorant: Identification and removal of type-checking bad smells*. In 2008 12th European Conference on Software Maintenance and Reengineering (pp. 329–331). IEEE. <https://doi.org/10.1109/CSMR.2008.4493333>
- Tokuda, L., & Batory, D. (2001). *Evolving object-oriented designs*

with refactorings. Automated Software Engineering, 8(1), 89–120.
<https://doi.org/10.1023/A:1008365906390>

- Mens, T., & Tourwé, T. (2004). *A survey of software refactoring*. IEEE Transactions on Software Engineering, 30(2), 126–139.
<https://doi.org/10.1109/TSE.2004.1265817>
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison- Wesley Professional.
<https://martinfowler.com/books/refactoring.html>
- Opdyke, W. F. (1992). *Refactoring Object-Oriented Frameworks* (Doctoral dissertation, University of Illinois at Urbana-Champaign).
<https://www.lri.fr/~mbl/ENS/CSCW/2009/papers/Opdyke-PhD.pdf>

